SQL Query Optimization Masterpad

A complete, intuitive guide to optimize your SQL queries with real-world examples.

Table of Contents

Fundamentals

What Makes a Query Slow?

SQL queries are slow because of: - **Full Table Scans** - Reading every row instead of using indexes - **Poor Join Strategy** - Joining large tables inefficiently - **Subqueries** - Running separate queries for each row - **Missing Indexes** - No indexes on WHERE, JOIN, or ORDER BY columns - **Data Type Mismatches** - Comparing different data types - **Excessive Functions** - Using functions on indexed columns

The 80/20 Rule

80% of performance gains come from: 1. Adding the right indexes 2. Fixing JOIN logic 3. Removing subqueries 4. Using proper WHERE clauses

---

Indexing Strategies

Understanding Indexes

An **index** is like a book's table of contents—it helps you find data quickly without reading every page.

Types of Indexes: - **Single Column Index** - Fast for filtering on one column - **Composite Index** - Fast for multiple columns (order matters!) - **Primary Key Index** - Automatically created, unique - **Unique Index** - Ensures no duplicate values

When to Create Indexes

Create indexes on: - Columns in WHERE clause - Columns in JOIN conditions (ON clause) - Columns in ORDER BY clause - Columns in GROUP BY clause - Foreign key columns

Avoid indexes on: - Columns with very few unique values (gender, status) - Columns with NULL values - Columns that are frequently updated - Small tables (<10,000 rows)

Index Examples

Slow Query - No Index:

```
SELECT * FROM Customers
WHERE email = 'john@example.com';
-- Scans entire table: 2 seconds
```

Fast Query - With Index:

```
-- Create index first
CREATE INDEX idx_customer_email ON Customers(email);

-- Same query now runs in 10ms
SELECT * FROM Customers
WHERE email = 'john@example.com';
```

Composite Index Example:

```
-- If you often filter by department AND salary
CREATE INDEX idx_dept_salary ON Employees(department_id, salary);

-- This query uses the index
SELECT * FROM Employees
WHERE department_id = 5 AND salary > 50000;

-- This query also uses index partially
SELECT * FROM Employees
WHERE department_id = 5;

-- This query CANNOT use index efficiently
SELECT * FROM Employees
WHERE salary > 50000;
-- Because salary is second in index!
```

Order Matters in Composite Indexes: Put columns that are **frequently filtered first**, then columns used for sorting.

Query Structure Optimization

1. Use SELECT Specific Columns (Not *)

Slow:

```
SELECT * FROM Orders WHERE status = 'Completed';
-- Fetches all 50 columns you don't need
```

Fast:

```
SELECT order_id, customer_id, total_amount, order_date
FROM Orders
WHERE status = 'Completed';
-- Fetches only 4 columns
```

Why? Less data transfer, better memory usage, faster network transmission.

---

2. WHERE Clause Placement & Order

Slow - Multiple Filters:

```
SELECT * FROM Orders
WHERE YEAR(order_date) = 2025
  AND status = 'Completed'
  AND payment_method = 'Credit Card';
-- Processes millions of rows
```

Fast - Most Restrictive First:

```
SELECT * FROM Orders
WHERE status = 'Completed'           -- Filters to 20% of rows
  AND payment_method = 'Credit Card' -- Filters to 5% of rows
  AND order_date >= '2025-01-01';    -- Applied last
-- Processes only relevant rows
```

Why? Database stops processing when criteria are met (short-circuit evaluation).

---

3. Avoid Functions on Indexed Columns

Slow - Function on Column:

```
SELECT * FROM Customers
WHERE YEAR(created_date) = 2025;
-- Cannot use index on created_date
```

Fast - Direct Comparison:

```sql
SELECT * FROM Customers
WHERE created_date >= '2025-01-01'
  AND created_date < '2026-01-01';
-- Uses index on created_date
```

Slow - String Manipulation:

```sql
SELECT * FROM Users
WHERE UPPER(email) = 'JOHN@EXAMPLE.COM';
-- Cannot use index
```

Fast - Exact Match:

```sql
SELECT * FROM Users
WHERE email = 'john@example.com';
-- Uses index
```

---

4. Use UNION Over OR (Sometimes)

Slow - Multiple ORs:

```sql
SELECT * FROM Products
WHERE category = 'Electronics'
   OR category = 'Books'
   OR category = 'Furniture';
```

Fast - Using IN:

```sql
SELECT * FROM Products
WHERE category IN ('Electronics', 'Books', 'Furniture');
```

Fast - Using UNION (for complex conditions):

```sql
SELECT product_id, name FROM Products
WHERE category = 'Electronics' AND price > 1000
UNION
SELECT product_id, name FROM Products
WHERE category = 'Books' AND price > 50;
-- Can use different indexes for each condition
```

Join Optimization

Join Order Matters

Principle: Join the **smallest** result set to the **largest**.

Slow - Wrong Order:

```sql
SELECT o.order_id, c.customer_name, p.product_name
FROM Orders o
JOIN OrderDetails od ON o.order_id = od.order_id      -- 1M rows
JOIN Customers c ON o.customer_id = c.customer_id      -- 100K customers
JOIN Products p ON od.product_id = p.product_id;       -- 50K products
-- Joining 1M rows to everything
```

Fast - Filtered First:

```sql
SELECT o.order_id, c.customer_name, p.product_name
FROM Orders o
WHERE o.order_date >= '2025-01-01'                     -- Filter: 100K rows
JOIN Customers c ON o.customer_id = c.customer_id      -- Join 100K to 100K
JOIN OrderDetails od ON o.order_id = od.order_id       -- Join result to details
JOIN Products p ON od.product_id = p.product_id;
-- Joining filtered sets
```

Types of JOINs & Performance

INNER JOIN - Fastest, eliminates non-matching rows

LEFT JOIN - Slower, keeps all left-side rows

FULL OUTER JOIN - Slowest, keeps all rows from both sides

Example - Choose Better JOIN:

```sql
-- Slow (LEFT JOIN keeps all customers even without orders)
SELECT c.customer_id, c.name, o.order_id
FROM Customers c
LEFT JOIN Orders o ON c.customer_id = o.customer_id;

-- Fast (INNER JOIN if you only need customers WITH orders)
SELECT c.customer_id, c.name, o.order_id
FROM Customers c
INNER JOIN Orders o ON c.customer_id = o.customer_id;
```

Avoid Multiple Joins on Large Tables

Slow - 4 Joins:

```sql
SELECT o.order_id, c.name, p.name, s.name, sh.name
FROM Orders o
JOIN Customers c ON o.customer_id = c.customer_id
JOIN OrderDetails od ON o.order_id = od.order_id
JOIN Products p ON od.product_id = p.product_id
```

```sql
JOIN Suppliers s ON p.supplier_id = s.supplier_id
JOIN Shipping sh ON o.shipping_id = sh.shipping_id;
```

Fast - Use CTE or Denormalization:

```sql
WITH OrderDetails_Enriched AS (
  SELECT
    o.order_id,
    o.customer_id,
    od.product_id,
    p.supplier_id
  FROM Orders o
  JOIN OrderDetails od ON o.order_id = od.order_id
  WHERE o.order_date >= '2025-01-01'  -- Filter early
)
SELECT
  ode.order_id,
  c.name,
  p.name,
  s.name
FROM OrderDetails_Enriched ode
JOIN Customers c ON ode.customer_id = c.customer_id
JOIN Products p ON ode.product_id = p.product_id
JOIN Suppliers s ON p.supplier_id = s.supplier_id;
```

Subquery & CTE Optimization

Problem: Correlated Subqueries

Slow - Runs for EVERY row:

```sql
SELECT customer_id, name,
  (SELECT COUNT(*) FROM Orders
   WHERE customer_id = Customers.customer_id) AS order_count
FROM Customers;
-- If 100K customers, subquery runs 100K times!
```

Fast - Use JOIN & GROUP BY:

```sql
SELECT c.customer_id, c.name, COUNT(o.order_id) AS order_count
FROM Customers c
LEFT JOIN Orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name;
-- Runs in one pass
```

Problem: Subqueries in WHERE

Slow - Subquery for each row:

```
SELECT * FROM Orders
WHERE customer_id IN (
  SELECT customer_id FROM Customers
  WHERE country = 'USA'
);
-- Database re-evaluates subquery for many rows
```

Fast - Use JOIN or EXISTS:

```
-- Option 1: JOIN
SELECT DISTINCT o.* FROM Orders o
JOIN Customers c ON o.customer_id = c.customer_id
WHERE c.country = 'USA';

-- Option 2: EXISTS (better for existence check)
SELECT o.* FROM Orders o
WHERE EXISTS (
  SELECT 1 FROM Customers c
  WHERE c.customer_id = o.customer_id
  AND c.country = 'USA'
);
```

Best Practice: Use CTEs for Readability & Performance

Using CTE (Common Table Expression):

```
WITH USA_Customers AS (
  SELECT customer_id, name, email
  FROM Customers
  WHERE country = 'USA'
),
Customer_Orders AS (
  SELECT c.customer_id, c.name, COUNT(o.order_id) AS order_count
  FROM USA_Customers c
  LEFT JOIN Orders o ON c.customer_id = o.customer_id
  GROUP BY c.customer_id, c.name
)
SELECT * FROM Customer_Orders
WHERE order_count > 5;
```

Benefits: - More readable - Easier to debug - Better performance (DB optimizes the entire flow) - Reusable within same query

---

Execution Plan Analysis

How to Read Execution Plans

SQL Server:

```
SET STATISTICS IO ON;  -- Enable statistics
SET STATISTICS TIME ON;

SELECT * FROM Orders WHERE order_date = '2025-01-15';

-- Output shows:
-- Table 'Orders'. Scan count = 1, logical reads = 100
-- CPU time = 15ms, elapsed time = 45ms
```

PostgreSQL:

```
EXPLAIN ANALYZE
SELECT * FROM Orders WHERE order_date = '2025-01-15';

-- Shows:
-- Seq Scan on orders (Slow - full table scan)
-- Index Scan using idx_order_date on orders (Fast)
```

MySQL:

```
EXPLAIN
SELECT * FROM Orders WHERE order_date = '2025-01-15';

-- Shows:
-- type: index_range_scan (Good)
-- type: ALL (Bad - full table scan)
-- rows: 1000 (Estimated rows examined)
```

Key Metrics to Watch

Metric	Good	Bad
Scan Type	Index Seek	Table Scan
Rows Examined	< 1000	> 100K
Execution Time	< 100ms	> 5000ms
CPU Time	Low	High
I/O Operations	Few	Many

Example: Interpreting Execution Plans

Bad Plan - Full Table Scan:

```
Seq Scan on Orders (cost=0.00..35000.00 rows=1000000)
```

```
  Filter: (order_date = '2025-01-15')
-- Read 1M rows to find 100
```

Good Plan - Index Seek:

```
Index Scan using idx_order_date (cost=0.00..50.00 rows=100)
-- Read only indexed data, much faster
```

Common Mistakes & Fixes

Mistake 1: Missing WHERE Clause

Slow:

```sql
SELECT * FROM Orders;
-- Loads entire table into memory: 500MB+
```

Fast:

```sql
SELECT * FROM Orders
WHERE order_date >= '2025-01-01';
-- Loads only recent orders: 50MB
```

Mistake 2: NOT IN with NULL Values

Slow & Broken:

```sql
SELECT * FROM Orders
WHERE customer_id NOT IN (
  SELECT customer_id FROM BlacklistCustomers
);
-- If BlacklistCustomers has NULL values, returns NO rows!
```

Fast & Correct:

```sql
SELECT * FROM Orders o
WHERE NOT EXISTS (
  SELECT 1 FROM BlacklistCustomers b
  WHERE b.customer_id = o.customer_id
);
```

Mistake 3: SELECT * with LIMIT

Slow:

```sql
SELECT * FROM Orders LIMIT 10;
-- Still loads all columns, just displays 10 rows
```

Fast:

```sql
SELECT order_id, customer_id, total_amount, order_date
FROM Orders
LIMIT 10;
-- Loads only needed columns
```

Mistake 4: Joining to Derived Tables

Slow:

```sql
SELECT o.order_id, summary.total
FROM Orders o
JOIN (
  SELECT customer_id, SUM(amount) AS total
  FROM Transactions
  GROUP BY customer_id
) summary ON o.customer_id = summary.customer_id;
-- Materializes entire temp table
```

Fast:

```sql
WITH Customer_Summary AS (
  SELECT customer_id, SUM(amount) AS total
  FROM Transactions
  GROUP BY customer_id
)
SELECT o.order_id, summary.total
FROM Orders o
JOIN Customer_Summary summary
  ON o.customer_id = summary.customer_id;
-- CTE is optimized by the database
```

Mistake 5: GROUP BY Without Filtering

Slow:

```sql
SELECT category, COUNT(*)
FROM Products
GROUP BY category;
-- Groups 1M products then filters
```

Fast:

```sql
SELECT category, COUNT(*)
FROM Products
WHERE price > 100  -- Filter first
```

```sql
GROUP BY category;
-- Groups only expensive products
```

Performance Checklist

Use this checklist before running queries in production:

Index Check

- ☐ All columns in WHERE clause have indexes
- ☐ All JOIN columns have indexes
- ☐ Composite indexes follow best order
- ☐ No unnecessary indexes on small tables

Query Structure Check

- ☐ Using SELECT with specific columns (not *)
- ☐ WHERE clause filters early and aggressively
- ☐ No functions applied to indexed columns
- ☐ UNION used instead of multiple ORs when beneficial

Join Check

- ☐ Using INNER JOIN instead of LEFT/FULL when possible
- ☐ Filtering large tables before joining
- ☐ Joining in order: small result → large result
- ☐ Not using > 4 joins without CTEs

Subquery Check

- ☐ No correlated subqueries
- ☐ Subqueries converted to JOINs where possible
- ☐ Using EXISTS instead of IN for existence checks
- ☐ Using CTEs for complex queries

Performance Check

- ☐ Ran EXPLAIN ANALYZE to review execution plan
- ☐ Logical reads are minimal
- ☐ Execution time is under acceptable threshold
- ☐ No table scans on large tables

Data Check

- ☐ Row count is reasonable for the business logic
- ☐ Data types match (no implicit conversions)

☐ NULL handling is correct
☐ No accidental duplicates in results

Quick Reference: Optimization Patterns

Pattern 1: Filter Before Join

```
WITH filtered_orders AS (
  SELECT * FROM Orders
  WHERE created_date >= '2025-01-01'
)
SELECT * FROM filtered_orders f
JOIN Customers c ON f.customer_id = c.customer_id;
```

Pattern 2: Aggregate Before Join

```
WITH order_summary AS (
  SELECT customer_id, COUNT(*) AS order_count
  FROM Orders
  GROUP BY customer_id
  HAVING COUNT(*) > 5
)
SELECT c.*, os.order_count
FROM Customers c
JOIN order_summary os ON c.customer_id = os.customer_id;
```

Pattern 3: Use Window Functions Instead of Correlated Subqueries

```
-- Instead of:
SELECT customer_id,
  (SELECT AVG(amount) FROM Orders) AS avg_order
FROM Orders;

-- Use:
SELECT customer_id,
  AVG(amount) OVER () AS avg_order
FROM Orders;
```

Pattern 4: Batch Operations

```
-- Instead of 1000 individual INSERT statements
-- Use:
INSERT INTO Orders (customer_id, amount)
VALUES
  (1, 100),
  (2, 200),
```

```
  (3, 300),
  ... (many more)
-- Reduces network roundtrips 100x
```

Remember: The Golden Rules

1. **Index First** - 80% of gains come from indexing
2. **Filter Early** - Apply WHERE clauses as early as possible
3. **Join Smart** - Use INNER JOIN, join in order: small→large
4. **Avoid Subqueries** - Convert to JOIN or EXISTS
5. **Measure Everything** - Use EXPLAIN ANALYZE before trusting speed
6. **Read Execution Plans** - They tell you exactly what's slow
7. **Test on Real Data** - Development data never matches production

---

Last Updated: January 2026
For: Data Analysts & Engineers
Next Step: Apply these patterns to your own queries and measure the difference!